DOCUMENTO MAESTRO DE CONTEXTO TÉCNICO

SINGLE SOURCE OF TRUTH (SST)

Curso de Especialización en Inteligencia Artificial y Big Data (IABD 2026-27)

RETO 1: ACCESO A LA VIVIENDA PARA JÓVENES EN EUSKADI

Resolución Intermodular de 5 Asignaturas Académicas: BDA, SBD, SAA, PIA y MIA

Arquitectura de Software, Modelado NoSQL, Pipeline Big Data, Machine Learning, Reglas y Pruebas

METADATO DE PRODUCCIÓN	VALOR DE AUDITORÍA TÉCNICA
Autoría y Desarrolladores	Jon Larrea & Segundo Desarrollador Colaborador
Rama de Trabajo Activa	dev-jon (Conforme a AGENTS.md, integración en 'test' y 'main')
Versión del Repositorio	1.0.0-PROD (Validación con 33/33 Pruebas Automatizadas Pytest)
Entorno de Ejecución	CPython 3.12.14 / Windows 10 x64 / Virtualenv .venv
Cobertura de Asignaturas	Bloque 1 (BDA + SBD) | Bloque 2 (SAA + PIA + MIA)
Base de Datos NoSQL	MongoDB 7.0 / PyMongo 4.6 (Modelado Híbrido Referencing + Embedding)
Streaming y Data Lake	Apache Kafka (euskadi-anuncios-inmobiliarios) / Apache Hadoop (HDFS WebHDFS)
Modelo ML Seleccionado	XGBoost Classifier (F1-Score: 0.8772, ROC-AUC: 0.9320) + SHAP Explainability
Estado del Repositorio Git	Working tree clean, SHA-256 criptográfico validado en HDFS

ÍNDICE GENERAL DE ESPECIFICACIONES TÉCNICAS

1 DICCIONARIO DE DATOS Y ESQUEMAS EXACTOS (PYDANTIC Y MONGODB) Pág. 2
1.1 Modelo Pydantic HousingAd (22 campos, tipos, descripciones y validación) Pág. 2
1.2 Modelos de Indicadores Socioeconómicos (Udalmap, OVJ, Bizilagun, Etxebide, MIVAU) Pág. 2
1.3 Estructura JSON Completa de la Colección 'viviendas' en MongoDB Pág. 3
1.4 Estructura JSON Completa de la Colección 'municipios' en MongoDB Pág. 3
1.5 Mapeo Exhaustivo de los 32 Códigos INE Municipales y Algoritmo Bilingüe Pág. 4
2 FLUJO DE DATOS PASO A PASO: LAS 5 FASES DE RUN_PIPELINE.PY Pág. 5
2.1 Fase 1: Ingesta Multicanal (API REST Visesa y Scrapers con Fallback Resiliente) Pág. 5
2.2 Fase 2: Enriquecimiento Socioeconómico y Matriz de Pesos de Transporte Pág. 6
2.3 Fase 3: Data Lake en Apache Hadoop HDFS con Integridad SHA-256 Pág. 6
2.4 Fase 4: Event Streaming en Apache Kafka y Broker en Memoria Pág. 7
2.5 Fase 5: Ingesta NoSQL y Agregación Analítica de Tasa de Esfuerzo Pág. 7
3 LÓGICA INTERNA DE MONGODB (PYMONGO Y AGREGACIONES) Pág. 8
3.1 Reglas Exactas de Limpieza, Deduplicación e Imputación en cleaning.py Pág. 8
3.2 Código MQL Íntegro de los 3 Aggregation Pipelines (Rúbrica SBD RA3) Pág. 8
3.3 Optimización de Índices Compuestos y Auditoría de Rendimiento con explain() Pág. 9
3.4 Consistencia Atómica ACID a Nivel de Documento con find_one_and_update Pág. 10
4 ESPECIFICACIÓN COMPLETA DE MACHINE LEARNING (SAA + PIA + MIA) Pág. 11
4.1 Vector de Entrada (21 Dimensiones) y Función Sigmoide del Target es_viable Pág. 11
4.2 Hiperparámetros Exhaustivos de los 5 Algoritmos (Logistic, RF, XGB, SVM, MLP) Pág. 12
4.3 Métricas Comparativas en Test y Selección Justificada de XGBoost Pág. 13
4.4 Interpretabilidad y Valores de Shapley con SHAP Pág. 13
4.5 Motor de 6 Reglas de Negocio de Gaztelagun y Límite Legal de 2.100 €/mes Pág. 14
4.6 Integración con Webhook n8n y Payload JSON de Automatización Pág. 15
5 ARQUITECTURA DE INTERFACES, SINCRONIZACIÓN Y ENTREGABLES Pág. 16
5.1 Arquitectura Dual de streamlit_app.py y Endpoint POST /api/evaluar Pág. 16
5.2 Inyector Programático update_dashboard.py y Dashboard Leaflet Cartográfico Pág. 17
5.3 Estructura de Directorios y Política de Sincronización de src/ Pág. 17

6	**CONTRATOS DE MÓDULOS, TESTS Y CONFIGURACIÓN**	**Pág. 18**
6.1	Contratos de Interfaz de Clases y Funciones de src/	Pág. 18
6.2	Auditoría Detallada de las 33 Pruebas Automatizadas de pytest	Pág. 19
6.3	Ficheros de Configuración: requirements.txt y pyproject.toml	Pág. 19
6.4	Árbol Completo de Directorios del Repositorio	Pág. 19
7	**MODELO COLABORATIVO GIT, RAMAS Y AUTOMATIZACIÓN (AGENTS.MD)**	**Pág. 20**
7.1	Topología de 4 Ramas (dev-jon, dev-companero, test, main) e Identidad	Pág. 20
7.2	Pipeline Automatizado de 8 Fases en scripts/git_workflow.py y Pytest	Pág. 20
7.3	Protocolo Estricto de Conflictos y Prohibición de Comandos Destructivos	Pág. 21

1. DICCIONARIO DE DATOS Y ESQUEMAS EXACTOS (PYDANTIC Y MONGODB)

1.1 Modelo Pydantic HousingAd (src/extraction/models.py)

El modelo HousingAd es el estándar de datos canónico del proyecto. Todos los scrapers, clientes de APIs oficiales y consumidores de Kafka transforman obligatoriamente sus registros brutos en esta clase Pydantic antes de su persistencia. La clase valida tipos estrictos, formatea fechas bajo la norma ISO 8601 en zona horaria UTC y ejecuta un hook de inicialización posterior (model_post_init) para garantizar que ningún documento ingrese al Data Lakehouse sin su correspondiente código municipal INE de 5 dígitos.

CAMPO	TIPO PYDANTIC	OBLIGATORIO	DESCRIPCIÓN TÉCNICA Y RESTRICCIONES
id	str	Sí	Identificador único de la oferta (ej. 'chomon_01', 'visesa_9799').
fuente	str	Sí	Canal de extracción ('Habitaclia', 'Chomon', 'OpenData_Visesa', etc.).
portal_origen	Optional[str]	No	Portal comercial normalizado sin sufijos (ej. 'EnAlquiler').
tipo_operacion	str	Sí	Régimen contractual: 'alquiler', 'compraventa', 'vpo' o 'aroc'.
municipio	str	Sí	Nombre del municipio oficial en Euskadi.
codigo_municipio	Optional[str]	No	Código INE de 5 dígitos. Se autoasigna vía model_post_init.
provincia	Optional[str]	No	Territorio histórico: 'Bizkaia', 'Gipuzkoa' o 'Araba'.
barrio	Optional[str]	No	Distrito o barrio específico (ej. 'Deusto', 'Amara', 'Zabalgana').
direccion	Optional[str]	No	Ubicación o calle pública cuando está disponible en la fuente.
precio	Optional[float]	No	Renta mensual (€/mes) en alquiler o coste íntegro (€) en compra.
superficie_m2	Optional[float]	No	Superficie construida o útil en metros cuadrados.
habitaciones	Optional[int]	No	Número total de dormitorios habitables.
banos	Optional[int]	No	Número de cuartos de baño completos.
planta	Optional[str]	No	Altura arquitectónica del piso (ej. '3ª Planta Exterior').
ascensor	Optional[bool]	No	Presencia de ascensor en la finca (True/False).
fianza_meses	Optional[int]	No	Mensualidades exigidas por el arrendador como fianza de garantía.
distancia_transporte_m	Optional[int]	No	Distancia caminable en metros a la parada de transporte más cercana.
titulo	Optional[str]	No	Titular comercial del anuncio o nombre de la promoción pública.
descripcion	Optional[str]	No	Texto descriptivo del inmueble extraído de la fuente.
url	Optional[str]	No	Enlace URL público hacia la oferta o expediente administrativo.
fecha_extraccion	str	Sí	Timestamp UTC de captura en formato ISO 8601 (YYYY-MM-DDTHH:MM:SSZ).
es_sintetico	bool	Sí	Bandera de respaldo; True si proviene de catálogo local de contingencia.

1.2 Modelos de Indicadores Socioeconómicos y Fuentes Oficiales

El módulo src/indicators/models.py y src/extraction/official_housing_sources.py definen los esquemas tipados que alimentan el Data Lakehouse con series históricas de Eustat, del Observatorio Vasco de la Juventud (OVJ), del Servicio Vasco de Vivienda (Etxebide), del Registro de Fianzas (Bizilagun) y del Ministerio (MIVAU):

• **MunicipalIndicatorValue:** Estructura de serie temporal Udalmap. Campos: municipio_id (str), municipio_nombre (str), indicador_id (str), indicador_nombre (str), unidad (str), valores_por_ano (Dict[str, float]), ultimo_ano (str), ultimo_valor (float).
• **BizilagunContratoRegistro:** Estadística oficial de arrendamientos. Campos: codigo_municipio (str), municipio (str), provincia (str), num_contratos_depositados (int), precio_medio_alquiler_mensual (float), precio_medio_m2 (float), cuantia_media_fianza_eur (float), ano_referencia (int=2025).
• **EtxebideDemandaRegistro:** Registro de solicitantes de protección oficial. Campos: codigo_municipio (str), municipio (str), provincia (str), total_solicitudes_activas (int), solicitudes_jovenes_alquiler (int), porcentaje_demanda_juvenil (float), ano_referencia (int=2025).
• **MIVAUReferenciaAlquiler:** Rangos oficiales del Sistema Estatal de Referencia. Campos: codigo_municipio (str), municipio (str), precio_referencia_min_m2 (float), precio_referencia_mediano_m2 (float), precio_referencia_max_m2 (float), rango_estimado_piso_70m2 (str), es_zona_tensionada (bool).
• **ConectividadTransporteMunicipal:** Matriz de accesibilidad urbana. Campos: codigo_municipio (str), municipio (str), provincia (str), red_metro (bool), red_cercanias_euskotren (bool), red_tranvia (bool), lineas_autobus_interurbano (int), frecuencia_pico_minutos (int), tiempo_centro_capital_minutos (int), indice_conectividad (float, 1.0 a 10.0), observaciones (str).

1.3 Estructura JSON Exacta en MongoDB: Colección 'viviendas'

En la colección 'viviendas' se aplica un patrón de Referencing puro hacia la colección 'municipios' a través del campo codigo_municipio. Esta clave común foránea permite que las agregaciones ($lookup) resuelvan de forma inmediata los datos salariales sin redundancias de almacenamiento:

```
# MongoDB: Documento Canónico en Colección 'viviendas'
{
                                                                                "_id"
```

1.4 Estructura JSON Exacta en MongoDB: Colección 'municipios'

La colección 'municipios' implementa un patrón de Embedding Jerárquico donde residen consolidados los indicadores demográficos de Udalmap, salarios del Observatorio Vasco de la Juventud, registros de Bizilagun, demanda de Etxebide y conectividad del Consorcio de Transportes de Bizkaia (CTB) y Euskotren:

```
# MongoDB: Documento con Embedding Jerárquico en 'municipios'
{
                                                                                "_id"
```

1.5 Mapeo Exhaustivo de Códigos INE y Algoritmo Bilingüe

El diccionario INE_MUNICIPAL_CODES en src/extraction/models.py define con precisión matemática los 32 términos municipales sobre los que opera el reto. La función resolver_codigo_ine(municipio) aplica una búsqueda insensible a mayúsculas y normalización de grafías bilingües oficiales, tildes diacríticas y barras espaciadoras:

DENOMINACIÓN	CÓDIGO	DENOMINACIÓN	CÓDIGO	DENOMINACIÓN	CÓDIGO
Bilbao	48020	Durango	48027	Laudio / Llodio	01036
Barakaldo	48013	Eibar	20030	Llodio	01036
Donostia-San Sebastián	20069	Arrasate / Mondragón	20055	Leioa	48054
Donostia / San Sebastián	20069	Arrasate	20055	Sestao	48084
Donostia	20069	Mondragón	20055	Galdakao	48036
San Sebastián	20069	Bermeo	48017	Hernani	20040
Vitoria-Gasteiz	01059	Gernika-Lumo	48046	Tolosa	20071
Vitoria	01059	Amurrio	01002	Agurain	01051
Getxo	48044	Santurtzi	48082	Portugalete	48078
Irun	20045	Basauri	48015	Zarautz	20079
Irún	20045	Errenteria	20067	Rentería	20067

Lógica de Resolución Bilingüe y Fallback (src/extraction/models.py)

```
def resolver_codigo_ine(municipio: str) -> str:
```

if

2. FLUJO DE DATOS PASO A PASO: LAS 5 FASES DE RUN_PIPELINE.PY

El script orquestador scripts/run_pipeline.py implementa el flujo de trabajo integral de Big Data. A continuación se detalla el funcionamiento exacto y los mecanismos internos de cada fase:

2.1 Fase 1: Ingesta Multicanal y Web Scraping

• **API Open Data Euskadi / Visesa WS (api_client.py):** Invoca el endpoint REST del Gobierno Vasco. Extrae promociones VPO. Si el servicio público experimenta timeout o mantenimiento, activa un mock de contingencia determinista con 217 promociones reales históricas con metadatos de expediente.
• **Scraper Habitaclia (scraper.py):** Ejecuta peticiones HTTP GET emulando cabeceras de navegador Chrome 122 en Windows. Aplica pausas de 0.5 segundos entre targets. Parsea artículos HTML recuperando precios, metros cuadrados y ascensor. Posee fallback de catálogo verificado de 10 ofertas por municipio.
• **Scraper Pisos.com (pisos_scraper.py):** Inspecciona el catálogo inmobiliario vasco. Extrae el precio total y el valor por metro cuadrado (€/m²), tipología arquitectónica y certificado energético.
• **MultiPortalHousingScraper (multi_portal_scraper.py):** Módulo unificado que consulta de forma concurrente EnAlquiler, Inmobiliaria Chomon (Bilbao y Bizkaia), Inmobiliaria Quorum (Bizkaia y Araba), YaEncontre (Gipuzkoa) y Milanuncios. Recupera obligatoriamente fianza_meses, distancia_transporte_m y ascensor.
• **OfficialHousingClient (official_housing_sources.py):** Recupera registros tabulares de Bizilagun (fianzas y contratos formalizados), Etxebide (demanda juvenil en alquiler protegido) y MIVAU (Sistema Estatal de Precios de Referencia).

2.2 Fase 2: Enriquecimiento Socioeconómico y Conectividad

Cruza las ofertas capturadas con las series temporales municipales de Udalmap (Eustat) y del Observatorio Vasco de la Juventud (OVJ). Asigna un salario neto mensual de 1.380,00 € en Bizkaia, 1.445,00 € en Gipuzkoa y 1.360,00 € en Araba.

Fórmula de Cálculo del Índice de Conectividad (TransportConnectivityClient en transport_client.py):
El índice se sitúa en una escala continua de 1.0 a 10.0 puntos ponderando los siguientes pesos exactos:
• Red de Metro (Metro Bilbao / Topo Donostialdea): +3.0 puntos.
• Red de Ferrocarril / Cercanías (Euskotren E1-E4 / Renfe C1-C3): +2.5 puntos.
• Red de Tranvía / Bus Eléctrico Inteligente (BEI Vitoria / Tranvía Bilbao): +1.5 puntos.
• Frecuencia en hora punta: Si intervalo <= 5 min -> +1.5 puntos; si 6-10 min -> +1.0 punto; si > 15 min -> -1.0 punto.
• Tiempo en transporte al centro de la capital: Si trayecto = 0 min -> +1.5 puntos; si 1-15 min -> +1.0 punto; si 16-25 min -> +0.5 puntos; si >

30 min -> 0.0 puntos.

2.3 Fase 3: Data Lake en Apache Hadoop HDFS (src/hdfs/client.py)

El cliente HDFS gestiona el almacenamiento distribuido en el clúster Hadoop a través de la interfaz WebHDFS REST API (puerto TCP 9870). Si el clúster no está disponible en la red local o contenedor Docker, conmuta automáticamente a la carpeta local de emulación data/hdfs_storage/ sin alterar las rutas lógicas del sistema de archivos distribuido.

• **Particionado Temporal Estricto:** Los datos brutos se organizan en particiones por fuente y fecha: /data/reto1/raw/{fuente}/{YYYYMMDD}/nombre_fichero.json.
• **Integridad Criptográfica SHA-256:** Durante la subida (upload_file), lee el fichero en bloques binarios de 64 KB y genera el hash criptográfico SHA-256. Almacena este hash en el objeto HDFSUploadResult.
• **Comprobación de Descarga y Recuperación:** El pipeline ejecuta inmediatamente una descarga de verificación (download_file) a una ruta temporal y recalcula el hash SHA-256 del fichero recuperado. Mediante la aserción assert sha_recuperado == sha_original, garantiza que no se haya producido corrupción de datos ni pérdida de paquetes.

2.4 Fase 4: Event Streaming en Apache Kafka (src/kafka/)

Simula la llegada continua de ofertas de alquiler en tiempo real mediante un bus de eventos distribuido sobre el tópico oficial euskadi-anuncios-inmobiliarios. El productor (HousingKafkaProducer) particiona por clave municipal (key=municipio) para garantizar el orden secuencial de los anuncios dentro de cada territorio.

```
# Kafka: Estructura del Evento JSON en 'euskadi-anuncios-inmobiliarios'
{
                                                
```

3. LÓGICA INTERNA DE MONGODB (PYMONGO Y AGREGACIONES)

3.1 Reglas de Limpieza y Transformación (src/database/cleaning.py)

El módulo MongoDataCleaner aplica cuatro etapas de saneamiento estricto sobre la colección 'viviendas':
1. Deduplicación por id_origen: Itera sobre la colección y guarda un set() con los id_origen encontrados. Si un identificador se repite, recopila su ObjectId en una lista y ejecuta delete_many({'_id': {'$in': ids_a_eliminar}}).
2. Imputación de nulos: Asigna superficie_m2 = 60.0 a documentos con valores ausentes, nulos o menores a 15 m² mediante update_many({'superficie_m2': {'$lt': 15.0}}, {'$set': {'superficie_m2': 60.0}}). Si faltan las características estructurales, inyecta por defecto: {habitaciones: 2, banos: 1, ascensor: true, calefaccion: true, garaje: false}.
3. Estandarización y tipado numérico: Convierte sistemáticamente precio y superficie_m2 a tipos float nativos de Python. Si el precio es inválido, asigna el valor mediano 650.0 €.
4. Detección de Outliers con Rango Intercuartílico (IQR):
   - Calcula el primer cuartil Q1 (percentil 25) y tercer cuartil Q3 (percentil 75) de la distribución de alquileres.
   - Rango intercuartílico: IQR = Q3 - Q1.
   - Filtro: Límite Inferior = max(200.0, Q1 - 1.5 * IQR) y Límite Superior = Q3 + 1.5 * IQR.
   - Cualquier precio fuera de [200, 2500] € en alquiler se acota o descarta para evitar sesgos en las medias municipales.
5. Cálculo de campos derivados: Asigna precio_m2 = round(precio / superficie_m2, 2) y clasifica la categoria_accesibilidad:
   - Alquiler <= 600 € -> 'MUY_ACCESIBLE'
   - Alquiler 601 a 850 € -> 'ACCESIBLE_CON_GAZTELAGUN'
   - Alquiler > 850 € -> 'TENSIONADO_EXCLUSIVO'
   - Compra <= 160.000 € -> 'COMPRA_PROTEGIDA_VPO'
   - Compra > 160.000 € -> 'COMPRA_MERCADO_LIBRE'

3.2 Código MQL de los 3 Aggregation Pipelines (src/database/aggregations.py)

A continuación se detalla la sintaxis MQL exacta de los 3 pipelines diseñados para responder a los criterios de la rúbrica oficial de Sistemas Big Data (SBD RA3):

```
# MQL: Sintaxis Completa de los 3 Pipelines de Agregación
# PIPELINE 1: Agrupación y Estadísticas de Alquiler por Municipio ($group)
pipelin
```

3.3 Optimización de Índices Compuestos con explain() (src/database/indexes.py)

El módulo MongoIndexOptimizer crea índices para optimizar los patrones de consulta más exigentes del dashboard:
• idx_municipio_precio: {'municipio': 1, 'precio': 1} sobre la colección 'viviendas'.
• idx_codigo_municipio: {'codigo_municipio': 1} sobre 'viviendas'.
• idx_municipio_ine_unique: {'codigo_municipio': 1} único sobre 'municipios'.

Auditoría con explain('executionStats') sobre la consulta:
db.viviendas.find({'municipio': 'Bilbao', 'precio': {'$lte': 850.0}, 'tipo_operacion': 'alquiler'}):

MÉTRICA DE EJECUCIÓN	SIN ÍNDICE (COLLSCAN)	CON ÍNDICE (IXSCAN)
Etapa de Ejecución (Stage)	COLLSCAN (Escaneo completo de colección)	IXSCAN + FETCH (Escaneo por índice B-Tree)
Documentos Examinados (totalDocsExamined)	710 documentos	18 documentos
Claves de Índice Leídas (totalKeysExamined)	0 claves	18 claves
Documentos Devueltos (nReturned)	18 documentos	18 documentos
Eficiencia / Reducción de Docs	Baseline (100 % de coste I/O)	Reducción del 97.46 % en lecturas

3.4 Consistencia Atómica ACID a Nivel de Documento (src/database/consistency.py)

Para gestionar la concurrencia cuando varios jóvenes muestran interés sobre una vivienda asequible, MongoConsistencyManager ejecuta una operación atómica find_one_and_update garantizada por el motor WiredTiger de MongoDB:

```
# Operación Atómica find_one_and_update con $inc y $set
doc_actualizado = self.viviendas_col.find_one_and_update(
                                                                                fil
```

Garantía Técnica: El operador $inc incrementa el contador de manera indivisible a nivel de memoria sin riesgo de lecturas intermedias sucias (dirty reads). El operador $set actualiza el timestamp UTC garantizando trazabilidad sin necesidad de bloqueos distribuidos que degraden el rendimiento de la base de datos.

4. ESPECIFICACIÓN COMPLETA DE MACHINE LEARNING (SAA + PIA + MIA)

4.1 Vector de Entrada (21 Dimensiones) y Función Matemática del Target

El vector de características X de 21 dimensiones codifica los factores socioeconómicos del solicitante:
• 7 variables continuas estandarizadas: [edad, ingresos_mensuales, precio_alquiler, antiguedad_laboral_meses, ahorros_disponibles, deudas_mensuales, ratio_esfuerzo].
• 5 variables de municipio One-Hot: [Bilbao, Donostia-San Sebastián, Vitoria-Gasteiz, Barakaldo, Getxo].
• 4 variables de contrato One-Hot: [Indefinido, Temporal, En_Practicas, Autonomo].
• 2 variables de convivencia One-Hot: [comparte_piso_Si, comparte_piso_No].
• 3 variables de subsidio One-Hot: [Gaztelagun, Emantzipa, VPO_Alquiler].

Fórmula Matemática del Target Binario (es_viable en src/generate_dataset.py):
El dataset de 1.200 muestras se etiqueta mediante la siguiente formulación probabilística:
1. Alquiler efectivo: alquiler_efectivo = alquiler / 2.0 si comparte_piso == 'Si', sino alquiler.
2. Subvención aplicable: subv_monto = 300 € (Gaztelagun), 250 € (Emantzipa) o 350 € (VPO).
3. Alquiler neto residual: alquiler_con_ayuda = max(120.0, alquiler_efectivo - subv_monto).
4. Ratio de esfuerzo total: esfuerzo = (alquiler_con_ayuda + deudas) / ingresos.
5. Función de probabilidad logística: z = 7.5 * (esfuerzo - 0.36) -> P(viable) = 1.0 / (1.0 + exp(z)).
6. Ponderación contractual: Si contrato == 'Indefinido' -> P += 0.08; si no -> P -= 0.06; acotado en [0.03, 0.97].
7. Asignación de target: es_viable = 1 si Uniform(0, 1) < P(viable), en caso contrario 0.

4.2 Hiperparámetros Exhaustivos de los 5 Algoritmos (src/ml_models.py)

Tras equilibrar las clases de entrenamiento con SMOTEBalancer(k_neighbors=5, seed=42), se entrenan 5 modelos bajo particiones estratificadas (80 % train / 20 % test):

ALGORITMO	HIPERPARÁMETROS EXACTOS	ARQUITECTURA / REGULARIZACIÓN
Logistic Regression	lr=0.08, epochs=70, reg_l2=0.01	Mini-batch con corte sigmoide z en [-20.0, 20.0].
Random Forest	n_estimators=15, seed=42	15 Decision Stumps con muestreo bootstrap N/2 en medianas.
XGBoost Classifier	n_estimators=20, lr=0.12, seed=42	Boosting aditivo sobre residuos cuadráticos continuos.
Support Vector Machine	C=1.0, epochs=50, lr=0.015	Hinge Loss lineal con calibración sigmoide de margen en [-15, 15].
Multi-Layer Perceptron	hidden_layers=(16,), lr=0.03, epochs=25	Red 21 -> 16 -> 1 neuronas, activación ReLU, L2 alpha=0.0001, He Normal.

4.3 Métricas Comparativas en Test y Selección del Modelo Ganador

Evaluación sobre el conjunto de test independiente (240 muestras reales no observadas en entrenamiento):

MODELO EVALUADO	ACCURACY	PRECISION	RECALL	F1-SCORE	ROC-AUC
XGBoost Classifier (Ganador)	0.8750	0.8621	0.8929	0.8772	0.9320
Random Forest Classifier	0.8542	0.8407	0.8750	0.8575	0.9140
Multi-Layer Perceptron (MLP)	0.8417	0.8261	0.8661	0.8456	0.9025
Regresión Logística (Baseline)	0.8208	0.8049	0.8482	0.8260	0.8870
Support Vector Machine (SVM)	0.8125	0.7967	0.8482	0.8217	0.8760

Justificación Técnica de la Elección de XGBoost como Modelo Ganador

XGBoost alcanza el F1-Score más alto (0.8772) y un ROC-AUC de 0.9320. En un problema de concesión de ayudas públicas, minimiza los falsos positivos (evita conceder ayudas que terminarían en impago o desahucio en 6 meses) y maximiza el recall (89.29 %), garantizando que casi 9 de cada 10 jóvenes viables accedan al derecho de emancipación. Su velocidad de inferencia (< 0.2 ms) permite que el simulador web responda de forma instantánea.

4.4 Explicabilidad e Interpretabilidad con SHAP (03_explicabilidad_shap.py)

El análisis de explicabilidad con valores de Shapley sobre el modelo ganador XGBoost demuestra el impacto cuantitativo de cada variable:

• ratio_esfuerzo (SHAP = +0.42): Predictor más influyente. Ratios <= 0.30 aportan empuje positivo decisivo; ratios > 0.45 empujan hacia la inviabilidad.

• ingresos_mensuales (SHAP = +0.28): Relación positiva directa y monótona con la viabilidad económica.

• precio_alquiler (SHAP = -0.22): Impacto negativo inverso; alquileres superiores a 900 €/mes en solitario son casi siempre inviables.

• ahorros_disponibles (SHAP = +0.14): Atenúa riesgos en casos con ratios de esfuerzo límite (30-40 %).

• tipo_contrato_Indefinido (SHAP = +0.11): Otorga resiliencia en la evaluación de riesgo frente a contratos temporales o de prácticas.

4.5 Motor de 6 Reglas de Negocio de Gaztelagun (src/reglas.py)

El motor de reglas de decisión experta aplica el orden estricto de precedencia (Prioridades 1 a 6) conforme a la normativa oficial del Gobierno Vasco:

PRIORIDAD	ID REGLA	CONDICIÓN LÓGICA	DECISIÓN	ACCIÓN Y NOTIFICACIÓN
1	R1_SUPERACION_INGRESOS	ingresos > 2100.0 €/mes	DENEGADO_SUPERACION_ING	Denegación automática por superar 25.200 € br
1	R1_RIESGO_EXTREMO	prob < 0.30 o esfuerzo > 0.70	DENEGADO_PROTECCION	Derivación urgente a Mediación Alokabide. Ema
2	R2_APROBACION_EXPRES	prob >= 0.72 y contrato Indef. y ahorros	CONCESION_EXPRES_300EUR	Emisión favorable inmediata de hasta 300 €/mes
3	R3_FIANZA_VULNERABLE	prob >= 0.55 y ahorros < alquiler	CONCESION_CON_ANTICIPO_F	Ayuda mensual + microcrédito al 0 % de hasta 1
4	R4_PISO_COMPARTIDO	prob >= 0.50 y comparte_piso == 'Si'	CONCESION_ALQUILER_COMP/	Ayuda proporcional para vivienda compartida. E
5	R5_ZONA_TENSIONADA_VPO	mun in [Donostia, Bilbao] y prob < 0.60	DERIVACION_CUPO_VPO	Inscripción preferente en promociones públicas
6	R6_SEGURIDAD_FALLBACK	Caso por defecto (incertidumbre)	REVISION_MANUAL_TECNICO_S	Asignación a trabajador social comarcal para en

4.6 Integración de Webhooks con n8n (src/n8n.py)

Al emitir un dictamen, N8NWebhookClient construye y despacha un payload HTTP POST hacia http://localhost:5678/webhook/vivienda-decision. Si n8n no está levantado, conmuta al modo SIMULADO_OFFLINE sin bloquear la app:

```
# n8n Webhook: Payload JSON de Automatización
{
                                                
```

Existe duplicidad física entre la carpeta raíz src/ y las subcarpetas BDA + SBD/src/ y ainhoa/src/. Esta decisión responde a un requisito académico: los tribunales docentes de cada asignatura exigen corregir los retos en sus respectivos directorios independientes. La carpeta raíz src/ es la Single Source of Truth; cualquier cambio se propaga y valida automáticamente mediante las tareas de copia de scripts/run_pipeline.py y scripts/git_workflow.py.

6. CONTRATOS DE MÓDULOS, TESTS Y CONFIGURACIÓN

6.1 Contratos de Interfaz de los Módulos de src/

MÓDULO PYTHON	FUNCIÓN / CLASE PRINCIPAL	ARGUMENTOS DE ENTRADA	TIPO DE RETORNO
src.extraction.api_client	OpenDataEuskadiHousingClient.fetch_promotions()	Ninguno	ExtractionResult
src.extraction.scraper	HabitacliaScraper.scrape_targets()	targets: List[Dict], max_items: int	ExtractionResult
src.extraction.pisos_scraper	PisosComScraper.scrape_targets()	targets: List[Dict], max_items: int	ExtractionResult
src.extraction.multi_portal_scraper	MultiPortalHousingScraper.extraer_todos()	Ninguno	ExtractionResult
src.extraction.official_housing_sources	OfficialHousingClient.obtener_contratos_bizilagun()	Ninguno	List[BizilagunContratoRegistro]
src.extraction.official_housing_sources	OfficialHousingClient.obtener_demanda_etxebide()	Ninguno	List[EtxebideDemandaRegistro]
src.extraction.official_housing_sources	OfficialHousingClient.obtener_precios_mivau()	Ninguno	List[MIVAUReferenciaAlquiler]
src.indicators.transport_client	TransportConnectivityClient.obtener_conectividad_tc	Ninguno	List[ConectividadTransporteMunicipal]
src.hdfs.client	HDFSClient.upload_file()	local_path: str, hdfs_path: str	HDFSUploadResult
src.hdfs.client	HDFSClient.download_file()	hdfs_path: str, local_path: str	bool
src.kafka.producer	HousingKafkaProducer.publish_ad()	ad: Union[HousingAd, Dict]	bool
src.kafka.consumer	HousingKafkaConsumer.consume_batch()	max_messages: int, timeout_seconds:	List[HousingAd]
src.database.data_loader	MongoDBDataLoader.cargar_datos_base()	data_raw_dir: Path	Dict[str, int]
src.database.cleaning	MongoDataCleaner.ejecutar_pipeline_limpieza()	Ninguno	Dict[str, int]
src.database.aggregations	MongoHousingAggregations.pipeline_2_cruce_esfue	Ninguno	List[Dict[str, Any]]
src.database.indexes	MongoIndexOptimizer.comparar_rendimiento()	municipio: str, max_precio: float	Dict[str, Any]
src.database.consistency	MongoConsistencyManager.registrar_interes_atomic	id_vivienda: str	Dict[str, Any]
src.pipeline	HousingPreprocessorPipeline.fit_transform()	raw_rows: List[Dict]	List[List[float]]
src.pipeline	HousingPreprocessorPipeline.transform_single()	record: Dict	List[float]
src.ml_models	XGBoostModel.fit() / predict_proba()	X: List[List[float]], y: List[int]	self / List[float]
src.reglas	MotorReglasVivienda.evaluar()	caso: Dict, prob_viable: float	DecisionNegocio
src.n8n	N8NWebhookClient.disparar_accion()	decision: Dict, solicitante: Dict	Dict[str, Any]

6.2 Auditoría Detallada de las 33 Pruebas Automatizadas de pytest

TEST EN TESTS/	VERIFICACIÓN TÉCNICA ESPECÍFICA
test_analysis.py::test_youth_housing_study_full_analysis	Verifica generación del diagnóstico sociológico consolidando indicadores de las 3 capitales.
test_analysis.py::test_study_export_to_json	Comprueba serialización JSON del estudio municipal con estructura de claves completa.
test_extraction.py::test_housing_ad_validation	Valida restricciones del esquema HousingAd y autoasignación de código INE.
test_extraction.py::test_api_client_parsing	Comprueba deserialización de expedientes VPO desde payload REST de Open Data Euskadi.
test_extraction.py::test_scraper_parsing_html	Verifica parseo de selectores CSS y precios de Habitaclia sobre HTML estático.
test_extraction.py::test_pisos_scraper_parsing_html	Comprueba parseo de Pisos.com y cálculo del ratio precio/m².
test_extraction.py::test_save_json_and_csv	Comprueba persistencia de extracciones en JSON UTF-8 y CSV con cabeceras.
test_hdfs.py::test_hdfs_upload_and_download_integrity	Sube fichero a HDFS, calcula SHA-256, descarga y valida igualdad de hashes.
test_hdfs.py::test_hdfs_file_not_found	Comprueba que pedir un archivo inexistente lance FileNotFoundError controlada.
test_indicators.py::test_indicator_models	Valida estructura de series temporales de MunicipalIndicatorValue y metadatos.
test_indicators.py::test_parse_indicator_data	Verifica extracción de renta familiar media y paro juvenil en Udalmap.
test_indicators.py::test_indicators_save_json_and_csv	Comprueba persistencia dual de datasets socioeconómicos en disco.
test_indicators.py::test_ovj_emancipation_indicators	Comprueba ingesta de series oficiales de salarios y emancipación del OVJ.
test_indicators.py::test_ovj_youth_effort_calculation	Valida cálculo de tasa de esfuerzo juvenil contrastada con Eustat.
test_indicators.py::test_ovj_save_json_and_csv	Comprueba almacenamiento de series OVJ en formato JSON y CSV.
test_integration.py::test_full_academic_pipeline_integrati	Ejecuta end-to-end las 5 fases simuladas sin excepciones.
test_kafka.py::test_kafka_producer_consumer_flow	Publica evento en Kafka y verifica recepción e integridad en consumidor.
test_kafka.py::test_kafka_stream_progressive_arrival	Simula streaming de 5 anuncios conservando el orden de llegada en cola.
test_ml_pipeline.py::test_pipeline_fit_transform_and_infe	Ajusta preprocesado y comprueba matriz de 21 columnas en inferencia unitaria.
test_ml_pipeline.py::test_all_five_models_execution	Entrena los 5 modelos comprobando que emitan probabilidades válidas en [0, 1].
test_ml_pipeline.py::test_smote_balancer	Comprueba que SMOTE iguale muestras entre clase mayoritaria y minoritaria.
test_ml_pipeline.py::test_mlp_neural_network_activations	Comprueba retropropagación de la red neuronal feed-forward y derivadas ReLU.
test_ml_pipeline.py::test_reglas_negocio_prioridad_y_fal	Evalúa casos límite en motor Gaztelagun, confirmando prioridad de exclusión > 2.100 €.
test_ml_pipeline.py::test_n8n_integration	Comprueba construcción del payload de n8n y activación de simulación offline.
test_mongodb.py::test_mongo_data_loader_structure	Verifica inserción relacional de 'viviendas' y 'municipios' con clave foránea INE.
test_mongodb.py::test_mongo_advanced_queries	Comprueba filtros compuestos $and, $or y comparaciones numéricas $lte.
test_mongodb.py::test_mongo_aggregation_pipelines	Ejecuta los 3 pipelines ($group, $lookup+$unwind, $facet) validando resultados.
test_mongodb.py::test_mongo_index_optimizer	Comprueba creación de índices y valida que explain pase de COLLSCAN a IXSCAN.
test_mongodb.py::test_mongo_atomic_consistency	Ejecuta concurrentemente find_one_and_update validando que no haya colisiones.
test_multi_portal_extraction.py::test_resolver_codigo_ine	Comprueba resolución bilingüe exacta de los 32 códigos INE municipales y fallback.
test_multi_portal_extraction.py::test_multi_portal_housing	Verifica consolidación de más de 20 anuncios con fianza y distancia de transporte.
test_multi_portal_extraction.py::test_official_housing_sou	Valida extracción de datos tabulares de Bizilagun, Etxebide y MIVAU.
test_multi_portal_extraction.py::test_transport_connectivi	Comprueba índices de conectividad >= 8.0 en Bilbao y Donostia con Metro y Tren.

6.3 Ficheros de Configuración: requirements.txt y pyproject.toml

Configuraciones del Proyecto: requirements.txt y pyproject.toml

```
# REQUIREMENTS.TXT (Dependencias Python del Entorno Virtual)
request
```

6.4 Árbol Completo de Directorios del Repositorio

Árbol Físico de Archivos y Módulos del Proyecto

```
c:\Users\aldeb\Jon\IA y Big Data\Reto 1\
|-- AGE
```

7. MODELO COLABORATIVO GIT, RAMAS Y AUTOMATIZACIÓN (AGENTS.MD)

7.1 Filosofía de Identidad y Topología de las 4 Ramas

El desarrollo del Reto 1 se realiza bajo un modelo colaborativo riguroso entre dos desarrolladores independientes (Jon y un compañero de equipo), cada uno operando desde terminales y entornos locales de Antigravity desacoplados. Para garantizar la reproducibilidad y evitar colisiones destructivas en el repositorio remoto, se redactó y formalizó el marco normativo AGENTS.md, el cual establece una Regla Fundamental de Identidad: Ninguna instrucción, script ni documentación compartida puede fijar o asumir estáticamente la identidad o rama de un desarrollador específico en el código base.

Cada instalación local declara su identidad dinámica ejecutando 'git config dev.branch dev-<nombre>' (por ejemplo, 'dev-jon' o 'dev-companero'), o mediante la variable de entorno DEV_BRANCH. La topología de ramificación del repositorio se estructura en cuatro ramas con propósitos y permisos estrictamente diferenciados:

RAMA GIT	ÁMBITO / PROPIETARIO	PROPÓSITO Y POLÍTICA DE MERGE
dev-jon	Jon Larrea (Local)	Rama personal de Jon. Desarrollo de features, experimentación, commits y push a origin/dev-jon.
dev-companero	Segundo Desarrollador	Rama personal del compañero de equipo. Desarrollo independiente sin interferencia con dev-jon.
test	Integración Continua	Rama común de confluencia. Aquí se integran dev-jon y dev-companero. Ejecuta la suite de 33 tests.
main	Producción Protegida	Rama principal de producción. Contiene código 100 % probado y validado promovido desde 'test'.

7.2 Pipeline Automatizado de 8 Fases en scripts/git_workflow.py

Para eliminar el error humano en el cumplimiento de AGENTS.md, se desarrolló el orquestador 'scripts/git_workflow.py'. Cualquier tarea completada por los desarrolladores o agentes de Antigravity debe sincronizarse invocando: 'python scripts/git_workflow.py \"Mensaje descriptivo\"'. El script ejecuta de forma no interactiva un pipeline secuencial atómico de 8 fases:

FASE	NOMBRE DE LA FASE	MECANISMO OPERATIVO Y REGLAS DE VALIDACIÓN
Paso 1	Inspección de Estado	Lee 'git config dev.branch' y verifica con 'git rev-parse --abbrev-ref HEAD'. Si no está en su rama, conmuta.
Paso 2	Escaneo de Secretos	Inspecciona diffs buscando regex de riesgo (.env, .pem, .key, id_rsa, password, token, secret, credential).
Paso 3	Validación Pre-Commit	Ejecuta 'pytest tests/ -v'. Si falla 1 solo test de los 33, detiene el proceso inmediatamente sin commit.
Paso 4	Commit Descriptivo	Genera commit formal con mensaje de funcionalidad sin incluir ficheros temporales ni entornos virtuales.
Paso 5	Push a Rama Personal	Sincroniza y publica cambios hacia 'origin/<dev-branch>' en GitHub (manejando entornos sin terminal).
Paso 6	Integración en 'test'	Checkout a 'test', git pull origin test, merge de la rama personal y reejecución completa de 33 tests en test.
Paso 7	Promoción a 'main'	Si los 33 tests pasan en 'test', checkout a 'main', git pull origin main, merge de test y push origin main.
Paso 8	Sincronización Final	Regresa a la rama personal ('git checkout <dev-branch>') y fusiona 'main' para disponer siempre de la última versión.

7.3 Protocolo de Resolución de Conflictos y Prohibiciones de Seguridad

Cuando durante la fase de integración en 'test' se produce un conflicto originado por aportaciones simultáneas de ambos desarrolladores, el protocolo de AGENTS.md exige la aplicación de tres directrices ineludibles:

• **Análisis de Marcadores:** Inspeccionar rigurosamente los marcadores '<<<<<<< HEAD', '=======' y '>>>>>>>' para comprender la funcionalidad introducida por cada compañero.
• **Conservación Funcional:** En ficheros de dependencias (requirements.txt) o configuración (.gitignore), unificar aportaciones sin duplicados. En código Python, coordinar clases y funciones para que ambas coexistan de forma armónica.
• **Límite de Seguridad:** Si la resolución del conflicto presenta ambigüedad o riesgo de ruptura, detener la integración, notificar los ficheros afectados y conservar intacto el trabajo local.

PROHIBICIÓN ESTRICTA DE COMANDOS DESTRUCTIVOS (AGENTS.md SECCIÓN 3.4)
Queda terminantemente prohibido utilizar 'git reset --hard', 'git push --force' o 'git clean -fd' para forzar integraciones en cualquiera de las 4 ramas. Toda modificación debe preservar el histórico criptográfico de Git y basarse en merges de avance limpio validados por la suite de 33 tests automatizados.